Making It Demoable

Immersion Week SF · Day 2 · Session 2.3

50

Minutes

You leave with a public URL

What This Session Covers

01 Wrap it so a stranger can drive

02 Ship it to a public URL

03 Write a three-minute script

04 Document what is faked

Could a stranger drive it?

From 2.2 to Now

A path

It runs when you run it

vs

A demo

It runs when they do

How We Get There

01

Wrap

A front end, however plain

02

Ship

A URL off your machine

03

Script

Three minutes, rehearsed

04

Admit

Write down what is fake

The Next 50 Minutes

11:00

Wrap it

A front end anyone can drive

11:20

Ship it

Public URL, tested cold

11:35

Script it

Three minutes, out loud

11:45

Confess

The caveat list, in the repo

"A demo nobody can open is a story."

- The Day 2 close

01

A Front End

Front end

The part that works without you in the room

What a Stranger Needs

To know what this thing does

Something obvious to click

A result they can read

No instructions from you

Two Honest Options

A Claude Artifact	A page you host
Minutes to build	More control
Shareable link included	More time to build
Good enough to judge	Yours after the week

Choosing Between Them

	Artifact	Hosted page
Time to live	**Minutes**	An hour or more
Control	Limited	Total
Demo risk	**Low**	Yours to manage

Claude Artifact

A working page Claude builds and hosts for you

Wrapping It, Either Way

01 Describe the one screen

Input, button, result — nothing else

02 Paste in a real result

So the layout fits actual output

03 Drive it yourself, cold

New tab, no context, no excuses

A wrapper worth asking for

One page. A box to paste a report in,
a Run button, and the result below it.
Show this exact output as an example:
<paste out/answer.md here>
Plain and readable. No styling flourish.

Put These on the Page

One sentence: what this does

An example input, pre-filled

The result, readable as text

A line saying what is faked

Leave These Out

Accounts and sign-in

Settings and options

A second page of any kind

Anything you would style twice

Three Tests of Drivable

One click to a result

From landing on the page, with nothing typed

Obvious what happened

The output reads without you explaining it

Survives a bad input

Says something, rather than showing nothing

Watch For These

An empty box with no hint

A result that needs your voice

A spinner that never finishes

Output only you can read

02

Off Your Laptop

What if the wifi drops?

Why Localhost Is Not a Demo

It dies when your laptop sleeps

Nobody can open it later

You cannot send it to anyone

Tomorrow's team cannot test it

Two Ways to Ship It

GitHub Pages

Already where your code is

A commit is a deploy

Static only, no server

Cloudflare

Workers run your code

Secret keys stay secret

A second account to make

Shipping with Pages

01 Put the page in the repo

index.html at the root, or in docs/

02 Turn Pages on

Settings, Pages, pick the branch

03 Wait, then open the URL

First build takes a minute or two

From repo to public URL

```
$ cp out/index.html docs/index.html
$ git add docs/index.html
$ git commit -m "publish demo page"
$ git push
# Settings > Pages > branch: main, /docs
```

What Breaks When You Go Public

Paths that only exist locally

A key that was in your shell

A file you never committed

Anything reading your disk

Check Before You Share It

No API keys in the page

No keys in the repo history

Nothing private in the fixture

Nothing you cannot show

Where does the data go?

One Way or Two

Static

A page that just shows

vs

Stateful

A page that remembers

Supabase

Hosted Postgres, with auth and storage

What You Get

Piece	What it gives you
Database	Rows your page can write
Auth	Sign-in you did not build
Storage	Files people upload
API	All of it, over HTTPS

Standing It Up

01 Create the project

One click, on the free tier

02 Make one table

The columns your demo needs

03 Copy the two keys

The project URL and the public key

Writing a row from a static page

```
const db = createClient(URL, PUBLIC_KEY)

await db.from("runs").insert({
  input: text,
  result: answer,
})
```

What a Demo Actually Stores

What a visitor typed in

The result you gave back

A file somebody uploaded

A count of how often it ran

The Two Keys Are Not the Same

The public key belongs in the page

The secret key never does

Turn row level security on first

Then check it as a stranger

Secrets Are Not Secret in a Page

Secrets keep it out of the repo

They do not keep it out of the page

A build that inlines it publishes it

Only server-side code keeps it private

Which Half Does What

Job	Which half
Serves the page	Pages or Cloudflare
Remembers input	Supabase
Handles sign-in	Supabase
Costs you nothing	Both

When Not to Bother

Your demo only ever reads

Nothing needs to outlive the tab

One saved result would do

Then stub it, like anything else

Who else can write to it?

Row level security

A rule for who may read or write each row

A New Table Is Wide Open

Row level security starts switched off

The publishable key sits in your page

So anyone can read every row

And anyone can write new ones

What Someone Could Do

Read everything you have stored

Insert a thousand junk rows

Fill the storage bucket up

Leave text the next visitor sees

Lock It Down

01 Turn the rule on

Before the URL is public, not after

02 Write one policy

Say exactly what a visitor may do

03 Try it as a stranger

Private window, and your own key

The smallest policy that works

```
alter table runs
  enable row level security;

create policy "visitors may submit"
  on runs for insert
  to anon with check (true);
-- no read policy: nobody gets it back
```

Then Write It Down

What the page is allowed to write

What it must never read back

Any personal data you now store

All of it into CAVEATS.md

Test It Like a Stranger

01 Open a private window

No logins, no cache, no extensions

02 Do nothing but click

Type nothing you have not pre-filled

03 Then try it on a phone

Half the people you send it to will

If You Are Behind

01 Ship what exists

An ugly live page beats a good local one

02 Drop the front end

A page showing one saved result still counts

03 Never drop the caveats

Five honest lines take two minutes

The Ship Checklist

Check	Pass means
Private window	It loads and runs
Phone	You can read the output
Laptop closed	A teammate can open it
No secrets	Nothing in page or repo

"Needs your laptop? Not shipped."

- Objective two, restated

03

Script and Caveats

Demo script

The three minutes, written down word for word

The Three Minutes

Time	What you say
0:00–0:30	The problem, concretely
0:30–2:00	Drive it, live, in silence
2:00–2:30	What is real, what is not
2:30–3:00	What you would build next

The First Thirty Seconds

Name who has the problem

Say what it costs them today

One sentence, no background

Then stop and show it

Do Not Say These

Let me explain the architecture

Normally this works

We ran out of time to

It is basically just a wrapper

Rehearse It Properly

01 Time it, out loud

Silently in your head is always shorter

02 Have someone else drive

The builder's hands hide the rough edges

03 Cut until it fits

Three minutes means three minutes

Caveat list

Everything stubbed, faked, or known to break

What Goes on the List

Every step you stubbed

Every hardcoded value

Inputs you know it fails on

Anything only tested once

How to Write It Down

What	Status
Login	Stubbed, one user
PDF parsing	One file only
Retrieval	Faked, four docs
Long inputs	Known to break

CAVEATS.md, in the repo root

What is real and what is not
- Login: stubbed, always user "ana"
- Input: only fixtures/report.pdf
- Retrieval: 4 docs, pasted not fetched
- Fails on: inputs over ~20 pages

The repo at the end of Day 2

```
happy-path/
├── docs/
│   └── index.html
├── fixtures/
├── steps/
├── run.py
├── CAVEATS.md
└── README.md
```

Hiding and Naming

Hiding a limit	Naming a limit
They find it anyway	You look in control
It looks like a mistake	It reads as a decision
You lose the room	The question moves on

What Day 3 Will Attack

The undocumented fake

Anything real-looking that is not on your list

The single input

The second thing they try will not be yours

The dead link

A URL that only worked on your machine

Leaving This Session

A URL a stranger can open

It works with your laptop shut

A three-minute script, rehearsed

CAVEATS.md, committed

Write This Down Now

Question	Your answer
The public URL	…
Who drives the demo	…
Biggest caveat	…
Most likely to break	…

Tonight, If You Have Energy

Send the link to one stranger

Watch where they get stuck

Fix only what stopped them

Add what broke to the caveats

What would you attack?

Next: The Battle Test

You design an attack, not a feature

Another team red-teams yours

Agents probe it, not just people

Bring the caveat list with you